

보안 인프라 포트폴리오

남재근

구축한 환경이 실제로 동작하는지 끝까지 확인합니다.

네트워크 구성, 접근통제, 웹 취약점 분석과 시스템 점검을 경험했습니다. 설정을 적용하는 데서 끝내지 않고 직접 시험하며 문제의 원인을 찾는 과정을 중요하게 생각합니다.

교육	가디언즈 정보보호 및 보안 인프라 운영 관리	프로젝트	팀 프로젝트 3회
수상	2차 프로젝트 우수상 · 창작 위게임 팀 1 위	Web	https://njgnet.github.io/

경험을 숫자로 정리했습니다.

3회

완료한 팀 프로젝트

67개

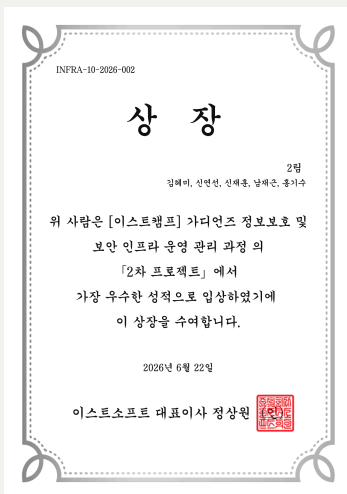
Ubuntu 시스템 점검 항목

2개

1차 본사·지사 시연 영상

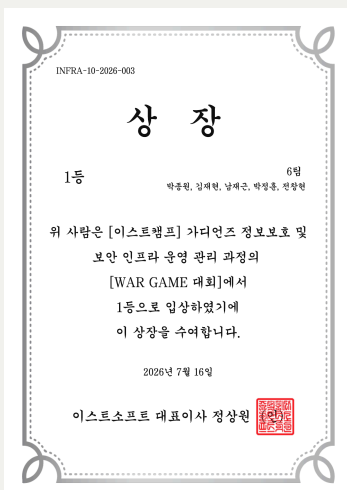
56장

직접 구성한 3차 발표자료



2차 프로젝트 우수상

취약점 분석·시스템 점검 프로젝트



창작 위게임 팀 1위

Challenge 09·10 제작 및 전 문제 해결

SIMMIT PAY 네트워크 구축

Packet Tracer 기반 통합 인프라와 ACL 정책 검증

본사와 지사 네트워크를 연결하고 VLAN과 라우팅을 확인한 뒤 ACL을 적용했습니다. VPN과 Dynamic NAT는 팀원과 협업했으며, 저는 ACL 설정과 전체 통신 검증에 가장 집중했습니다.

핵심 담당

ACL 정책·통신 테스트

검증 범위

본사·지사·재택근무 3개 환경

증거 자료

본사·지사 시연 영상 2개

라우팅 [OSPF]

```
router ospf 10
log-adjacency-changes
network 192.168.100.0 0.0.0.255 area 0
network 1.1.1.0 0.0.0.255 area 0
network 10.10.10.0 0.0.0.255 area 0
```

본사 라우터 [OSPF 10]

```
router ospf 20
log-adjacency-changes
network 192.168.200.0 0.0.0.255 area 0
network 3.3.3.0 0.0.0.255 area 0
network 10.10.10.0 0.0.0.255 area 0
network 2.2.2.0 0.0.0.255 area 0
```

지사 라우터 [OSPF 20]

```
router ospf 30
log-adjacency-changes
network 1.1.1.0 0.0.0.255 area 0
network 2.2.2.0 0.0.0.255 area 0
```

외부 라우터 [OSPF 30]

```
router ospf 15
log-adjacency-changes
network 3.3.3.0 0.0.0.255 area 0
network 192.168.150.0 0.0.0.255 area 0
```

재택근무(영업) 라우터 [OSPF 15]

본사·지사 및 서비스 구간을 분리한 Packet Tracer 네트워크

어려웠던 점

정책 한 줄로 정상 통신까지 차단됐습니다.

처음에는 통신이 되지 않을 때마다 ACL부터 수정해 원인을 구분하기 어려웠습니다.

Easy Hotel 취약점 분석과 시스템 점검

6건

웹 취약점 재현·조치

9 → 0

조치 전후 취약 항목

67개

시스템 점검 항목

64장

최종 통합보고서

2. 모의해킹 결과

2.1. 총평

본 시스템에 대한 보안 진단 결과, 총 6건의 주요 취약점이 식별되었습니다. 진단 항목은 크게 '인증', '회원 관리', '시스템 접근' 영역으로 구분되며, 각 항목에서 발견된 기술적 결함들은 시스템의 전반적인 보안 무결성과 데이터 보호 체계에 중대한 위험 요소로 작용하고 있습니다.

주요 진단 결과:

- 인증 및 세션 제어 취약점 (3.1.1 ~ 3.1.3):**
SQL 인젝션을 통한 인증 우회 가능성이 확인되었으며, Open Redirection 및 세션 고정(Session Fixation) 취약점이 도출되었습니다. 이는 공격자가 정상적인 인증 절차를 회피하거나, 탈취한 세션 정보를 이용하여 타인으로 위장 접속할 수 있는 심각한 위험을 내포하고 있습니다.
- 회원 정보 관리 및 인가 제어 취약점 (3.2.1 ~ 3.2.2):**
거장형 XSS(Stored XSS) 취약점을 통해 악의적인 스크립트 실행 및 사용자 정보 탈취가 가능하며, 무작위한 인가(IDOR) 결함으로 인해 타인의 예약 정보를 열람하거나 임의로 조작할 수 있는 상황입니다. 이는 개인정보 보호 및 서비스 데이터 무결성에 치명적인 영향을 줄 수 있습니다.
- 시스템 접근 제어 취약점 (3.3.1):**
SSRF(Server-Side Request Forgery) 취약점이 식별되어, 외부에서 시스템 내부 자원으로 비인가 요청을 수행할 가능성이 확인되었습니다. 이를 통해 서버 내부 인프라에 대한 추가적인 공격 시도가 우려됩니다.

결론 및 제언:

현재 본 시스템은 인증 우회, 데이터 조작, 서버 내부 접근 등 핵심 보안의 핵심 요소들이 복합적으로 존재하는 상태입니다. 각 항목별 발견된 취약점은 단독으로도 시스템의 보안을 무력화할 수 있는 수준이므로, 개별 단계에서의 보안 코딩 리듬 및 검증 절차를 통해 발견된 모든 취약점에 대한 즉각적인 조치가 요구됩니다.

2.2 결과 요약

No	대상	취약점	위치
1		SQL 인젝션	http://easyhotel.com/member_login.php
2	로그인 페이지	Open Redirection	http://easyhotel.com/member_login.php
3		세션 고정	http://easyhotel.com/member_login.php
4	회원가입 페이지	Stored XSS	http://easyhotel.com/member_register.php
5	예약 상세 페이지	IDOR	http://easyhotel.com/member_reservation_detail.php
6	메인 페이지	SSRF	https://easyhotel.com/main.php

모의해킹 결과보고서에 정리한 6개 취약점

취약점 검증

공격 조건을 만들고 조치 후 같은 방법으로 재시험했습니다.

SI를 활용해 취약 코드 초안을 구성했지만 그대로 사용하지 않고, 실제 환경에서 공격이 재현되는 이유와 코드 변경 후 차단 여부를 확인했습니다.

문서 작성

모의해킹 결과와 시스템 점검 내용을 정리했습니다.

취약점의 원인, 재현 조건과 조치 결과를 모의해킹 결과보고서에 정리하고 시스템 점검·최종보고서 작성에도 참여했습니다.

점검 스크립트의 틀을 먼저 시험했습니다.

01

어려웠던 점

67개 항목을 나누어 작성하기 전에 공통 출력 형식과 집계 기준이 필요했습니다.

02

초기 틀 제작

멘토님의 피드백을 바탕으로 시로 초안을 잡고 Ubuntu에서 명령어와 경로를 직접 확인했습니다.

03

팀 공유

U-01~U-67 실행 결과가 한 화면에 집계되는 형태를 캡처해 팀원들에게 먼저 보여줬습니다.

04

재검증

양호 58개·취약 9개에서 설정을 수정한 뒤 67개 전체 양호를 확인했습니다.

```
=====
점 검 요 약      전 체 : 67      양 호 : 양 호 58      취 약 : 취약 9 수 동 확 인 : 수 동 확 인 0
=====
```

조치 전 · 양호 58 / 취약 9

```
=====
점 검 요 약      전 체 : 67      양 호 : 양 호 67      취 약 : 취약 0 수 동 확 인 : 수 동 확 인 0
=====
```

조치 후 · 67개 전체 양호

제가 먼저 보여준 부분

점검 함수의 형태와 결과 집계 방식을 먼저 실행해 본 뒤 “이런 방식으로 합치면 될 것 같다”고 공유했습니다. 각 팀원이 작성한 항목을 같은 형식에 넣을 수 있도록 공통 출력 함수와 실행 순서를 정리했습니다.

채택 여부와 별개로 직접 동작까지 확인했습니다.

CRON · TTY OUTPUT TEST

예약 실행 결과를 활성화된 Ubuntu 터미널에서 바로 확인하는 방식

점검 결과를 로그 파일에 저장하는 것에 더해, 관리자가 별도 명령어를 입력하지 않아도 현재 열려 있는 터미널에서 결과를 확인할 수 있으면 좋겠다고 생각했습니다.

실행 시간을 지정하고 활성 터미널로 결과를 보내는 방법을 직접 시험했습니다. 팀의 최종 운영 방식으로 채택되지는 않았지만, 화면에 결과가 나타나는 과정까지 확인해 시연 영상으로 남겼습니다.

[시연 영상 열기 ↗](#)

2.2 보안 점검 자동화 및 상시 모니터링 체계

보안 점검 자동화 및 상시 모니터링 체계 구축

본 시스템은 보안 점검의 누락을 방지하고 일관된 보안 수준을 유지하기 위해, 자동화 스크립트를 활용한 정기 점검 체계를 구축하였습니다.

- **자동화 구현:** 리눅스의 cron 서비스를 활용하여 매일 오후 1시에 보안 점검 스크립트(security_check.sh)가 자동으로 실행되도록 설정하였습니다.
- **설정 이유:** 본 시스템은 호텔의 체크인(15:00) 및 체크아웃(11:00) 업무가 최소화되는 매일 오후 1시를 보안 점검 자동화 시간으로 설정하였습니다. 이는 서비스 가용성을 최우선으로 고려한 조치이며, 점검 결과를 당일 업무 시간 내에 확인하여 신속한 사후 조치가 가능하도록 운영 효율성을 극대화하였습니다.
- **상시 기록 체계:** 점검 결과는 /var/log/security_report_[날짜].log 파일로 자동 저장되며, 이를 통해 관리자가 별도의 수동 작업 없이도 시스템의 보안 상태를 상시로 확인하고 추적할 수 있도록 하였습니다.
- **보안 유지:** 매일 수행되는 자동 점검을 통해 기존에 조치 완료한 67개 항목이 '양호' 상태로 지속 유지되고 있음을 검증하고 있으며, 이상 징후 발생 시 즉각적으로 인지할 수 있는 운영 체계를 확립하였습니다.

2.3 보안 자동화의 전략적 이점 및 설계 원칙

본 자동화 체계는 단순한 효율을 넘어, 시스템 운영의 안정성과 보안 무결성을 최우선으로 고려하여 설계되었습니다.

1) 자동화 도입 효과 및 사용자 이점

- **운영 효율성 극대화:** 기존의 수동 점검 방식(항목당 5~10분 소요, 휴먼 에러 발생 가능성)을 배제하고, Cron을 통한 자동 실행으로 리소스 소모를 99% 이상 절감하였습니다.
- **상시 가시성 확보:** 점검 시점 외에도 설정 변경 사항을 즉시 감지하여 선제적 위험 대응이 가능합니다.
- **컴플라이언스 대응 최적화:** KISA 가이드라인 기반의 결과 보고서를 즉시 생성하여, 향후 외부 보안 컨설팅 및 정기 점검 시 대응 준비 시간을 대폭 단축했습니다.

2) 셸 스크립트(Shell Script) 채택의 보안 전략

보안 자동화 도구 자체가 새로운 취약점이 되는 상황을 방지하기 위해 다음과 같은 원칙을 적용하였습니다.

- **최소 권한 및 순정 가능 활용:** 파이썬 등 외부 프로그램 설치로 인한 '배경 프로세스 취약점' 발생을 원천 차단하기 위해, 리눅스 순정 가능한 셸 스크립트만을 사용하였습니다.
- **검증 가능성 및 유연성:** 코드가 직관적이며서 보안 담당자가 즉시 악성 코드 여부를 검증할 수 있으며, 새로운 취약점 발견 시 즉각적인 코드 수정과 반영이 가능합니다.
- **스크립트 파일 무결성 관리:** 스크립트 자체의 보안을 위해 파일 권한을 chmod 700(또는 600)으로 설정하여, 관리자 외 비인가자의 접근을 원천 차단하였습니다.

PrivaDrive 기업 정보보호 진단

7종

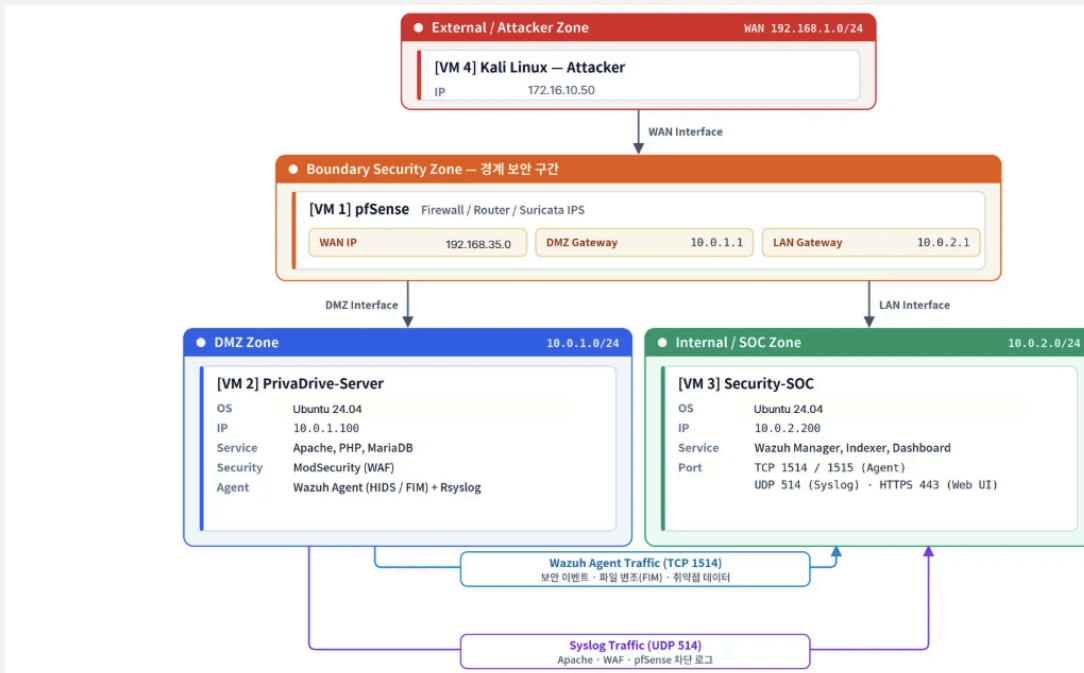
웹 취약점 점검

4종

악성파일 분석 도구

56장

직접 작성한 발표자료



External·Boundary·DMZ·SOC 구간으로 구성된 기업 인프라

취약점 조치

공격 결과와 발생 조건을 확인하고 조치 과정에 참여했습니다.

애플리케이션 코드와 보안 설정에서 수정할 부분을 정리하고, 조치 후 재시험 결과를 확인했습니다.

발표자료 단독 작성

팀 산출물을 56장 발표 흐름으로 다시 구성했습니다.

계획서, 구성도, 모의해킹, 탐지·차단, 로그와 악성코드 분석 결과를 목적→점검→발견→조치 순서로 정리했습니다.

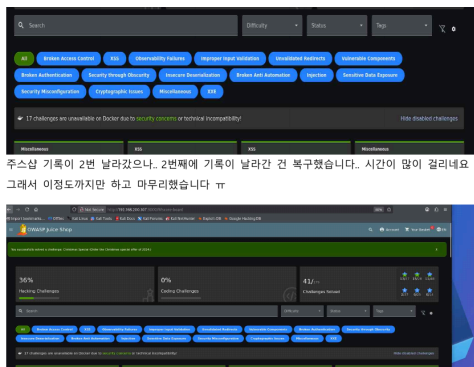
문제를 만들고 직접 풀어봤습니다.



TEAM CTF

문제 출제 의도와 워크스루 작성

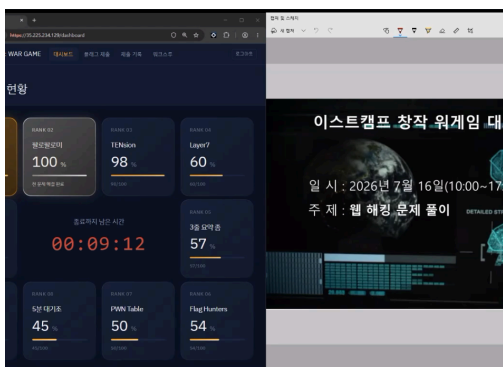
2팀 EasyHajo로 참가해 14개 플래그를 획득하고 팀 5위를 기록했습니다.



OWASP JUICE SHOP

요청값이 서버에서 처리되는 과정을 확인

관리자 인증 우회, 별점 요청값 변조와 관리자 권한 필드 추가를 실행하고 원인과 대책을 보고서로 작성했습니다.



CREATIVE WARGAME

Challenge 09·10 제작 · 팀 1위

보안 설정 오류와 Typosquatting 공급망 공격 문제를 제작했으며, ::6 팀으로 전체 10개 문제를 해결했습니다.

배운 내용을 실습으로 확인했습니다.

01

네트워크·인프라

Linux, Cisco, VLAN, Routing, ACL, VPN, NAT, Windows Server

02

보안 운영·관제

pfSense, Suricata, Snort, Wazuh, 로그 분석과 시스템 점검

03

웹 취약점 실습

DVWA, WebGoat, OWASP Juice Shop, Metasploit, WAF

04

문제 해결 방식

환경 구성 → 결과 확인 → 원인 분리 → 조치 → 같은 조건으로 재시험

PORTFOLIO LINKS

상세 화면과 원본 자료는 웹 포트폴리오에서 확인할 수 있습니다.

<https://njgnet.github.io/>

<https://github.com/njgnet>

PDF 출력 안내

이 문서는 웹 포트폴리오의 주요 경험과 근거 자료를 A4에 맞게 재구성한 출력본입니다. 프로젝트 원본 보고서와 시연 영상은 각 상세 페이지의 링크에서 확인할 수 있습니다.